Database System and Query Language

Lecture 4: E/R Model

February 8, 2026

Readings

- Database Systems: The Complete Book (2nd edition)
  - Chapter 4: High-Level Database Models (4.1- 4.6)
- Fundamental of Database Systems (7th edition)
  - Chapter 6,7

Lecture Objectives

1. **E/R Model Fundamentals:** Understand the core constructs of the Entity-Relationship model: entities, attributes, and relationships.
2. **Design Principles & Trade-offs:** Analyze design choices and constraints to accurately model real-world domains.
3. **From Conceptual to Logical:** Master the process of translating a high-level E/R diagram into a concrete relational database schema.
4. **Advanced Modeling Concepts:** Explore more complex scenarios using weak entities, specialization, and generalization (if time permits).

The Role of Conceptual Modeling

Why start with an abstract model?

- **Blueprint for Design:** The E/R model serves as a formal blueprint, allowing developers and stakeholders to agree on the database structure before implementation begins.
- **Communication Tool:** It provides a common language for both technical (developers, DBAs) and non-technical (domain experts, clients) parties to discuss data requirements.
- **Focus on Semantics:** It forces designers to consider the core meaning of the data:
  - What are the fundamental objects (*entities*) in our domain?
  - How do these objects interrelate (*relationships*)?
  - What business rules (*constraints*) govern these objects and their interactions?

The Goal

To create a high-level, implementation-agnostic model of a system's data requirements.

The Database Design Lifecycle

1. Requirements Analysis: The "What"
2. Conceptual Design: The "Blueprint"
3. Logical & Physical Design: The "How"

The Database Design Lifecycle

1. Requirements Analysis: The "What"
2. Conceptual Design: The "Blueprint"
3. Logical & Physical Design: The "How"

1. Requirements Analysis

This crucial first step involves gathering information from all stakeholders to understand the system's needs.

- **Interviews:** What data do you need to store? What questions will you ask of the data?
- **Existing Artifacts:** Analyzing forms, reports, and spreadsheets can reveal hidden entities and attributes.
- **Use Cases:** Defining user stories helps identify required data and access patterns. (e.g., "As a registrar, I need to enroll a student in a course.")

The Database Design Lifecycle

1. Requirements Analysis: The "What"
2. Conceptual Design: The "Blueprint"
3. Logical & Physical Design: The "How"

2. Conceptual Design

Translate the collected requirements into a formal, high-level data model.

- A precise description of entities, relationships, and constraints.
- This is the primary role of the **E/R Model**.
- **Crucially, this model is independent of the target database technology.** It does not care if you will eventually use a relational DB, a graph DB, or a document store.

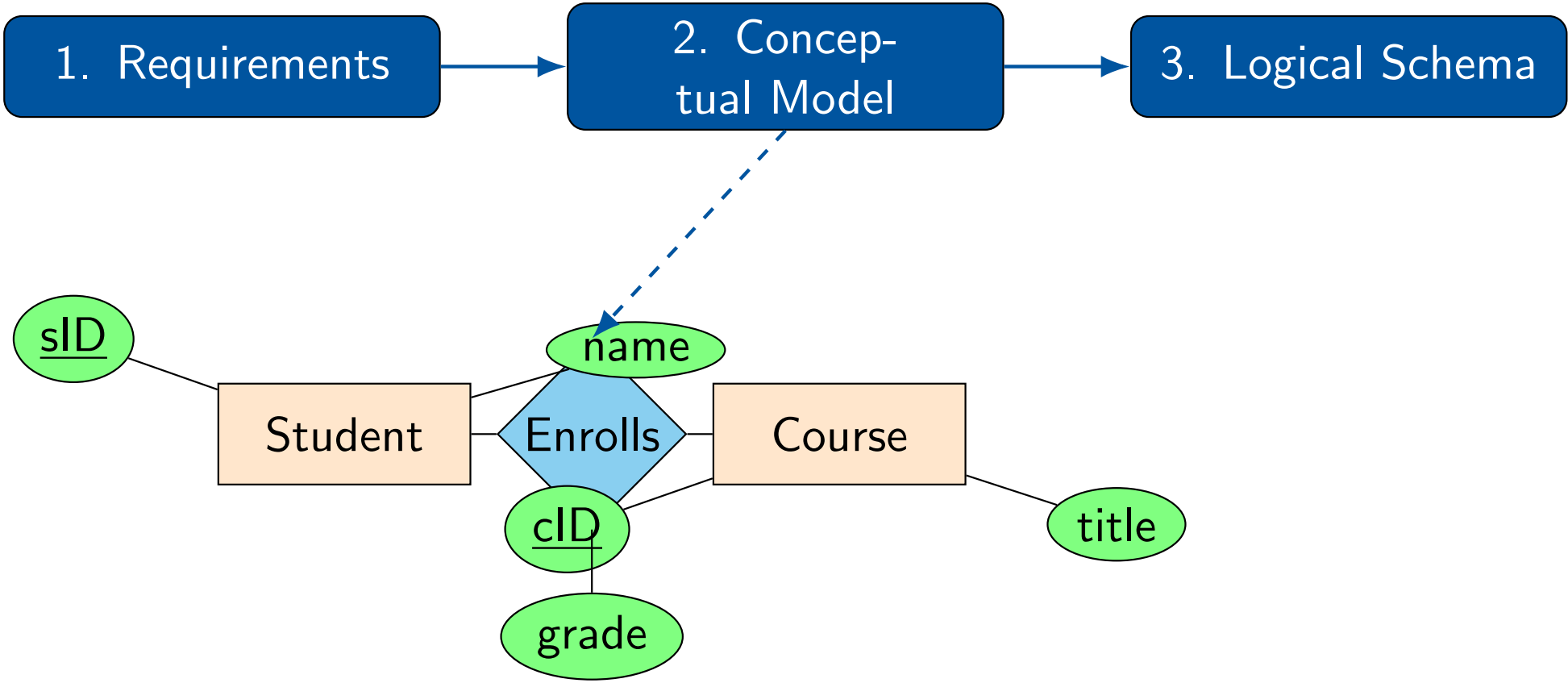
The Database Design Lifecycle

1. Requirements Analysis: The "What"
2. Conceptual Design: The "Blueprint"
3. Logical & Physical Design: The "How"

3. Implementation-Specific Design

- **Logical Design:** Map the conceptual model (E/R Diagram) to the specific data model of your chosen DBMS. For this course, this typically means translating the E/R diagram into a **relational schema** (a set of table definitions).
- **Physical Design:** Decide on the low-level implementation details: storage structures, indexing strategies, data partitioning, etc., to optimize performance and storage.

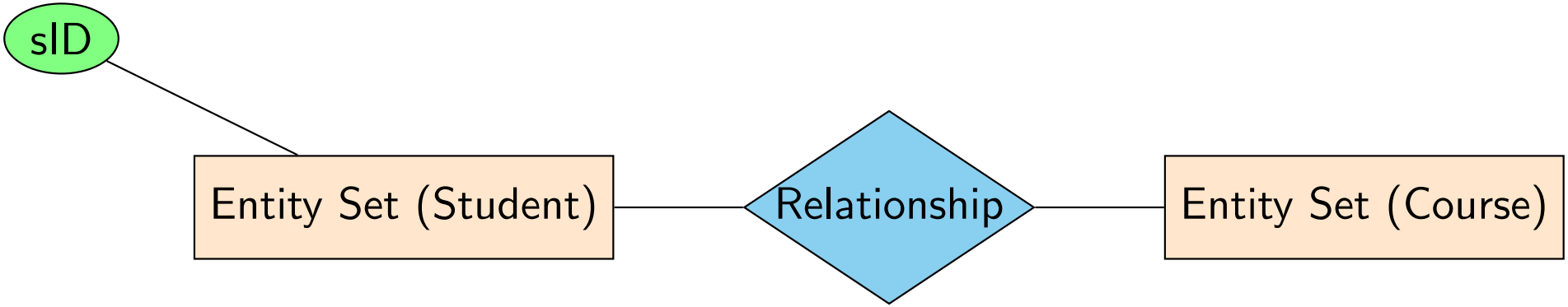
The Design Process: A University Example



The E/R model serves as the **bridge** between ambiguous user requirements and a concrete, implementable database schema.;

1. Core Components of the E/R Model

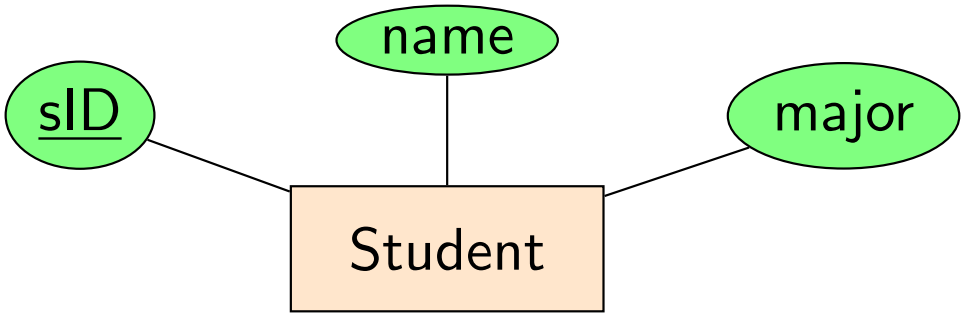
E/R Diagram Notation



Legend: Rectangles = Entities, Ellipses = Attributes,
Diamonds = Relationships, Underline = Key,
Double-line = Total Participation

The 'E' in E/R: Entity Sets

- **Entity**: A distinguishable real-world object or concept. *(e.g., a specific student, John Doe)*.
- **Entity Set**: A collection of similar entities. It acts as a *type* or a *class*. *(e.g., the set of all students at a university)*.
  - Represented by a **rectangle**.
- **Attribute**: A property that describes an entity from an entity set. *(e.g., Student ID, Name)*.
  - Represented by an **oval**.



Schema vs. Instance

An E/R Diagram models the **schema** (the structure), not the actual data instances.

Entity Sets vs. Individual Entities

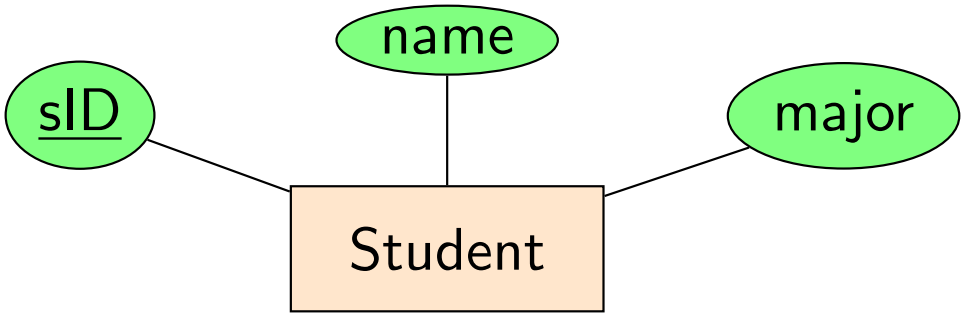
Key Distinction

The E/R diagram defines the *template* (the Entity Set). The actual data consists of many *instances* (Entities) that fit this template.

Entities (The Data Instances)

sID	name	major
111	Alice	CS
123	Bob	ECE
256	Carol	BME
...	...	...

Entity Set (The Schema)



These are **Entities**

Entities vs. Entity Sets: The Abstraction

Real-World Instances (Entities)

Specific objects with concrete values.

Entity 1

Name: Xbox
Category: Multimedia System
Price: $250

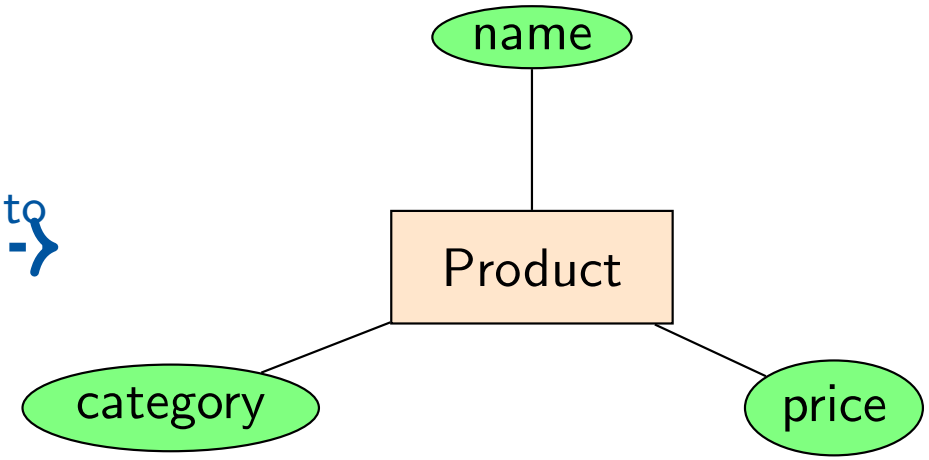
Abstracted to

Entity 2

Name: My Little Pony Doll
Category: Toy
Price: $25

E/R Model (Entity Set)

The abstract container (Schema).



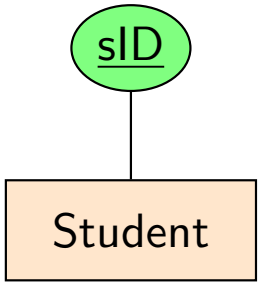
The E/R diagram models the **Entity Set** (the rectangle). The specific **Entities** are the data that populate the database.

Identifying Entities: Keys

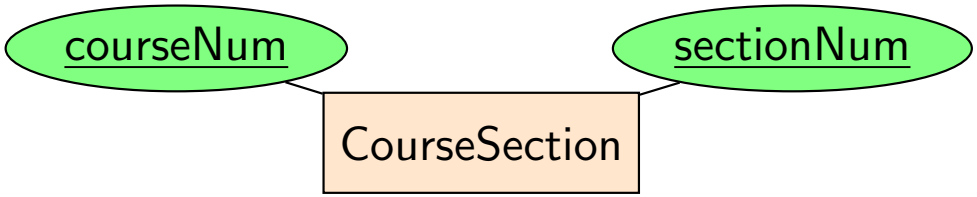
For an entity set to be valid, we must be able to uniquely identify every entity within it.

- **Superkey**: Any set of attributes that uniquely identifies an entity.
- **Candidate Key**: A *minimal* superkey.
- **Primary Key**: The candidate key chosen by the designer to be the main identifier for the entity set. Denoted by an <u>underline</u>.

Single-Attribute Key:



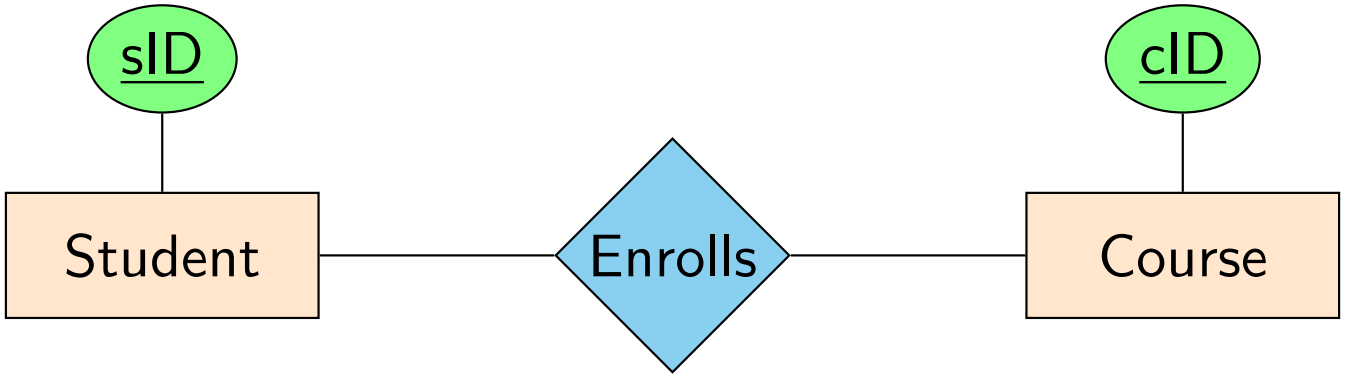
Composite Key:



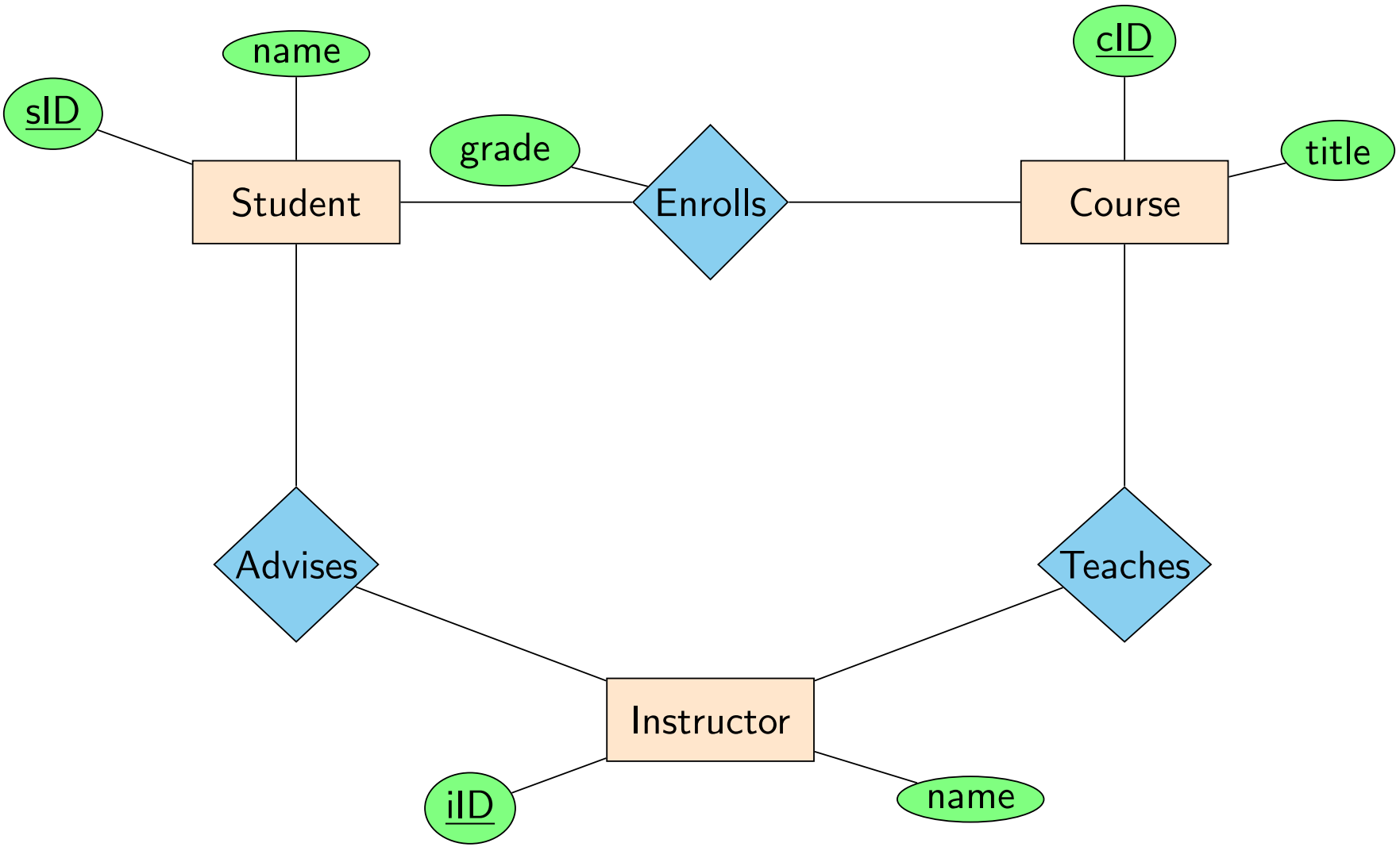
What makes a good primary key? Should we use "natural" keys (like a name) or create artificial "surrogate" keys (like a student ID)? Natural keys can change, which causes update anomalies. Surrogate keys are stable but require management.

The 'R' in E/R: Relationship Sets

- A **relationship** is an association among two or more entities.
  - e.g., Student '111' is enrolled in course 'EAS 550'.
- A **relationship set** is a collection of similar relationships.
  - Represented by a **diamond**.
- The number of participating entity sets defines the **degree** of the relationship set. The vast majority are binary (degree 2).



Example: A University Schema

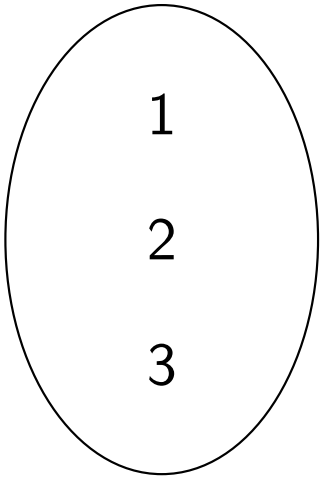


What is a Relationship?

A mathematical definition

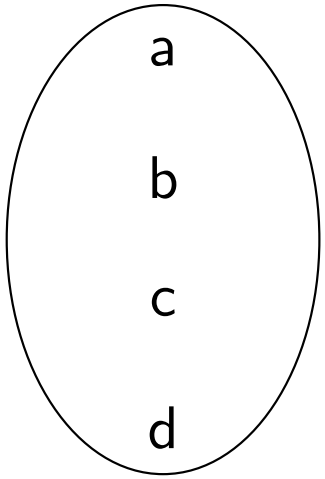
- Let A, B be sets

A=



$A = \{1, 2, 3\}$

B=



$B = \{a, b, c, d\}$

Relationships as Mathematical Sets

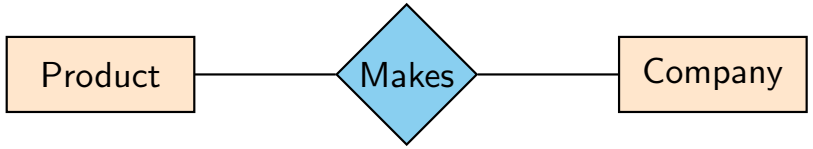
Entity Set: Company (C)

name	stockprice
GizmoWorks	25.00
GadgetCorp	40.00

Entity Set: Product (P)

name	category	price
Gizmo	Electronics	19.99
GizmoLite	Electronics	14.99
Gadget	Toys	9.99

Relationship Set: Makes



Definition: Makes $\subseteq C \times P$

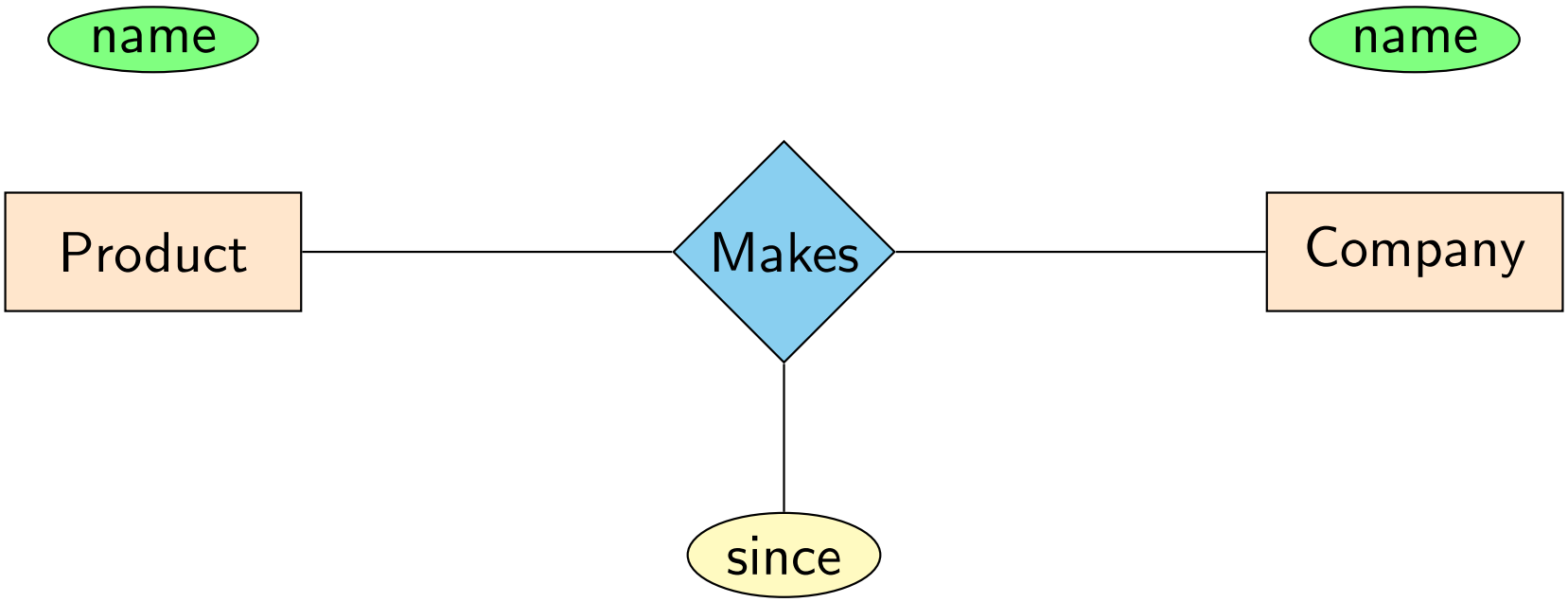
C.name	P.name
GizmoWorks	Gizmo
GizmoWorks	GizmoLite
GadgetCorp	Gadget

Observation

The relationship table contains pairs of keys. Note that "GizmoWorks" appears twice because it makes two distinct products.

Attributes on Relationships

Some data describes the *interaction* between entities, rather than the entities themselves.



Why not on Company?

The Company may have started making the "Gizmo" in 2010 and the "Gadget" in 2012. One date

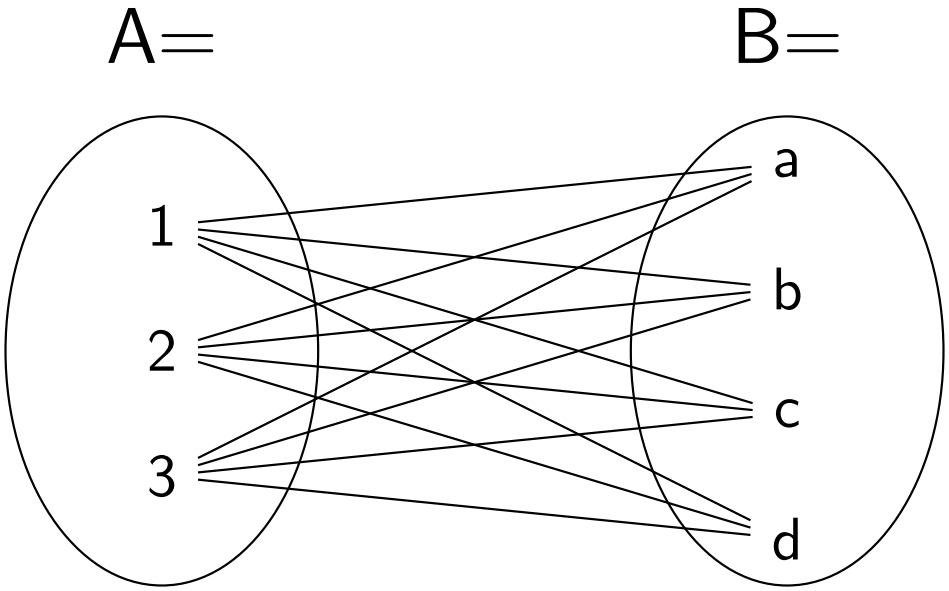
Why not on Product?

If both Company A and Company B make the "Gizmo", they might have started at different times.

What is a Relationship?

A mathematical definition

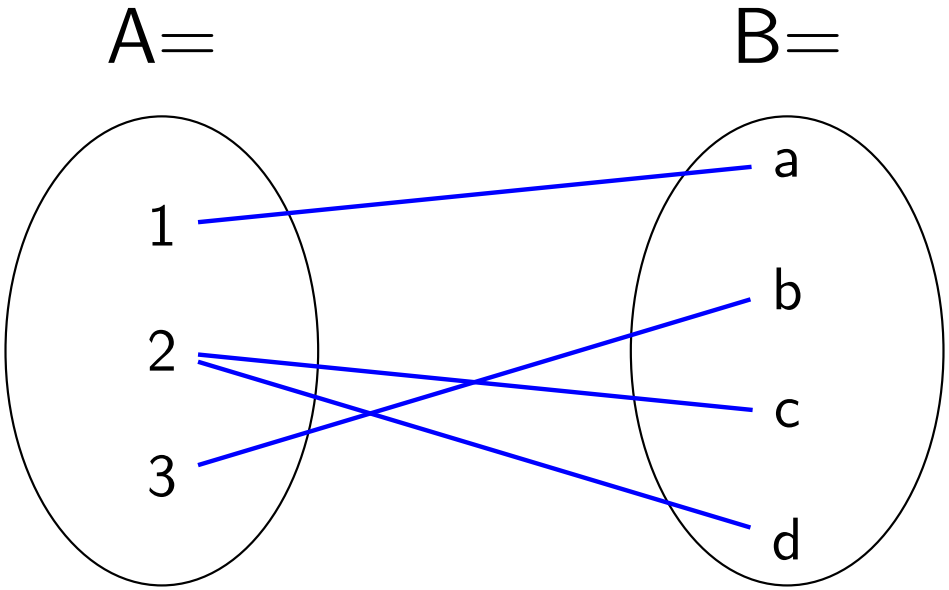
- Let A, B be sets: $A = \{1, 2, 3\}$, $B = \{a, b, c, d\}$
- $A \times B$ (the cross-product) is the set of all pairs (a,b)



$A \times B = \{(1, a), (1, b), (1, c), (1, d), (2, a), (2, b), (2, c), (2, d), (3, a), (3, b), (3, c), (3, d)\}$

What is a Relationship?

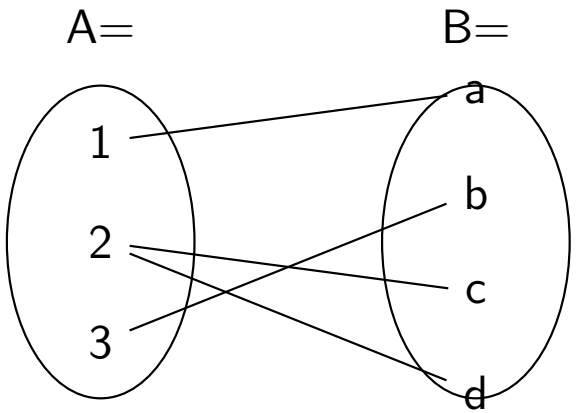
- Let A, B be sets: $A = \{1, 2, 3\}, B = \{a, b, c, d\}$
- $A \times B$ (the cross-product) is the set of all pairs (a,b)
- We define a **relationship** to be a **subset** of $A \times B$



$$R = \{(1, a), (2, c), (2, d), (3, b)\}$$

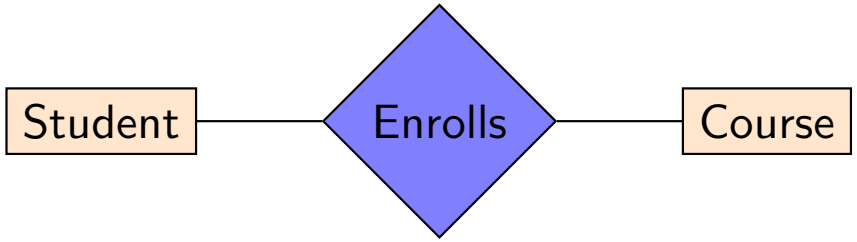
What is a Relationship?

- Let A, B be sets
- $A \times B$ (the cross-product) is the set of all pairs
- A **relationship** is a **subset** of $A \times B$



Example

Enrolls is a relationship: it is a subset of Student $\times$ Course.



The Mathematics of Relationships

- Let $E_1, E_2, \ldots, E_n$ be entity sets.
- A **relationship set** R is a subset of the Cartesian product of these sets:

$$R \subseteq E_1 \times E_2 \times \cdots \times E_n$$

- Each element $r \in R$ is a tuple $(e_1, e_2, \ldots, e_n)$, where each $e_i \in E_i$. This tuple is called a **relationship instance**.

The Mathematics of Relationships

- Let $E_1, E_2, \ldots, E_n$ be entity sets.
- A **relationship set** R is a subset of the Cartesian product of these sets:

$$R \subseteq E_1 \times E_2 \times \cdots \times E_n$$

- Each element $r \in R$ is a tuple $(e_1, e_2, \ldots, e_n)$, where each $e_i \in E_i$. This tuple is called a **relationship instance**.

Applying to our University Schema

The *Enrolls* relationship set is a subset of *Student* $\times$ *Course*. An instance could be:

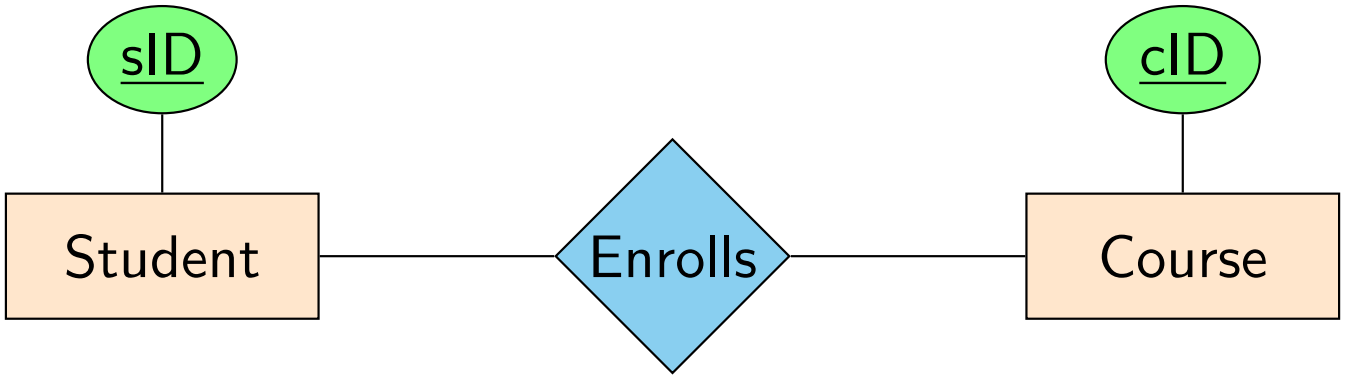
`(<student_entity_111>, <course_entity_EAS550>)`

Core Implication

Because a relationship set is mathematically a **set**, any given tuple of entities can appear at most **once**. You cannot enroll the same student in the exact same course twice within the set.

The Key of a Relationship Set

- Since a relationship instance is defined by the entities it connects, the **primary key** of a relationship set is the **union** of the primary keys of the participating entity sets.



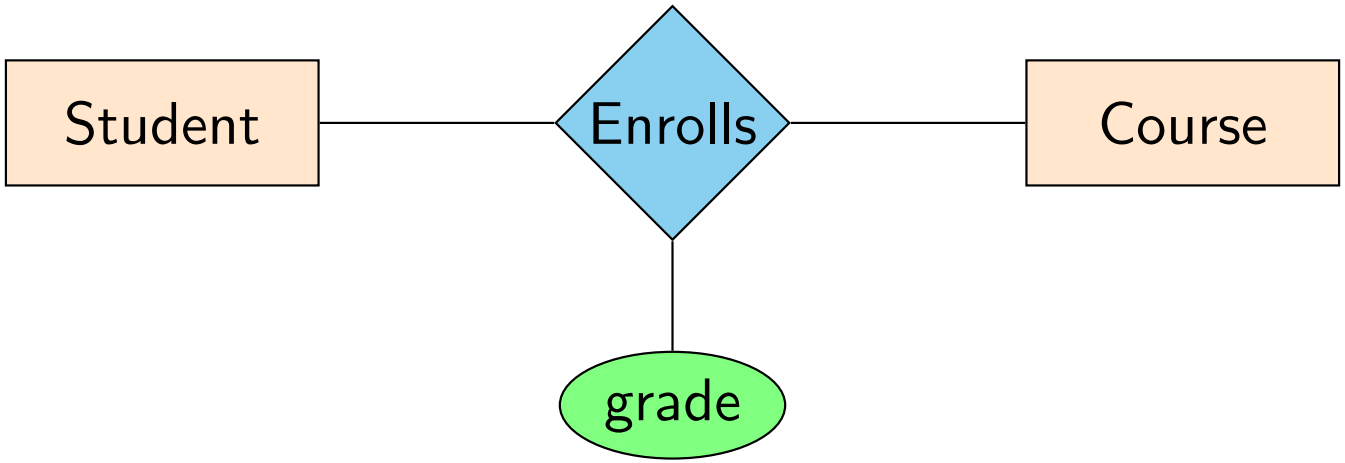
The primary key for the *Enrolls* relationship is {sID, cID}.

Consequence

This structure inherently prevents duplicate entries for the same student-course pair.

Attributes on Relationships

- Sometimes, a piece of data describes the *association* between entities, not the entities themselves. These are modeled as attributes on the relationship set.



Design Question

Why isn't 'grade' an attribute of 'Student'?

Design Question

Why isn't 'grade' an attribute of 'Course'?

A student has a different grade for each course they take, and a course has different grades for each student enrolled. The 'grade' only makes sense in the context of a *specific enrollment*.

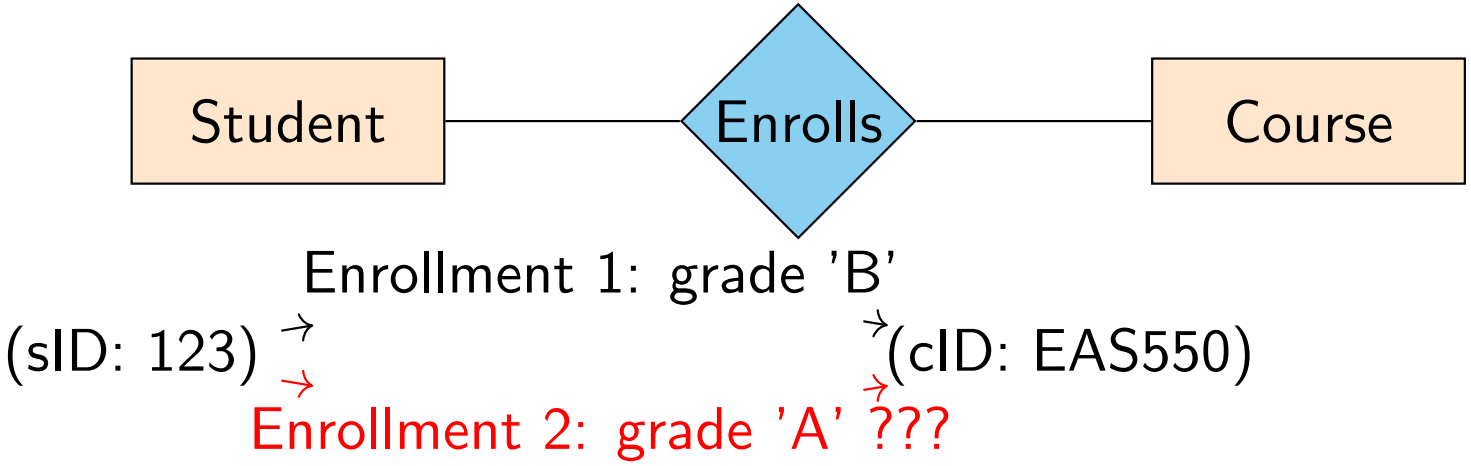
Common ER Misconceptions

- Why **Grade** is not an attribute of Student or Course
  - A student has different grades per course.
  - A course has different grades for each student.
- Why **Address** may be an Entity, not just an Attribute
  - Multiple employees may share the same address.
  - An employee may have multiple addresses over time.

Design Challenge: When Uniqueness is Wrong

Business Rule: "A student is allowed to retake a course to get a better grade."

Problem: Our current model cannot handle this.

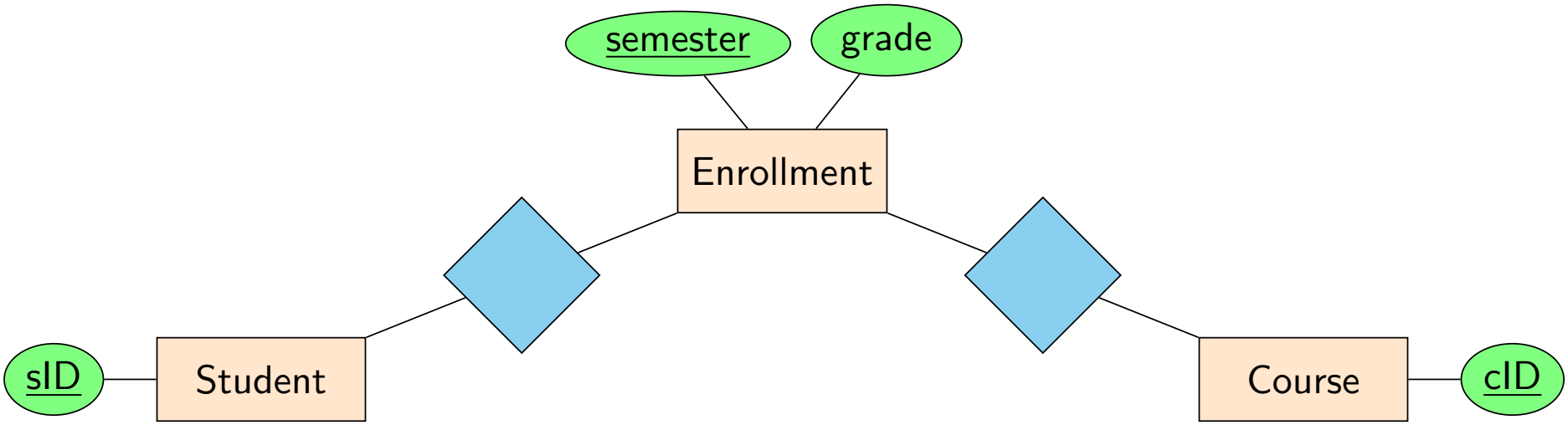


The Conflict

Since the *Enrolls* relationship set is keyed by {sID, cID}, the pair (123, EAS550) can only appear once. How can we model this correctly?

Solution: Promoting a Relationship to an Entity

To model multiple instances of an interaction between the same entities, we can convert the relationship set into an entity set. This is often called an **associative entity**.



How it Works

The new 'Enrollment' entity's key must include more than just the student and course. By adding semester, we can now uniquely identify each attempt: (sID, cID, semester).

2. Design Principles & Constraints

Cardinality Constraints (Multiplicity)

Cardinality constraints define the number of entities in one set that can be associated with an entity from another set.

- **Many-to-Many (N:M):** The default. An employee can work on many projects; a project can have many employees.
- **One-to-Many (1:N):** A department can have many employees; an employee belongs to at most one department.
- **One-to-One (1:1):** A department is managed by at most one employee; an employee manages at most one department.

Notation: The Arrow

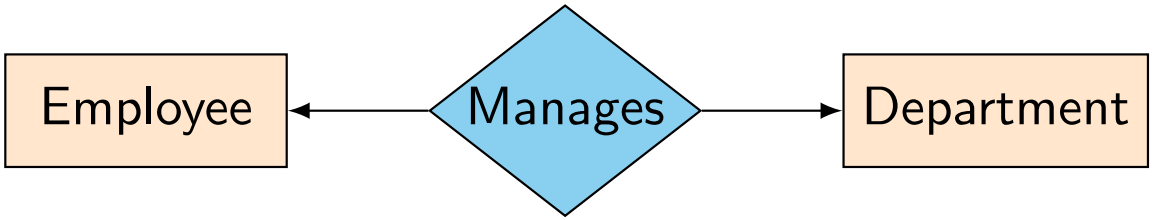
An arrow $\rightarrow$ from a relationship to an entity set E means "at most one." Each entity from the *other* participating sets can be related to at most one entity in E.

Cardinality Examples

Many-to-Many (N:M)

One-to-Many (1:N)

(An employee works in at most one department.)



One-to-One (1:1)

(An employee manages at most one department, and vice-versa.)

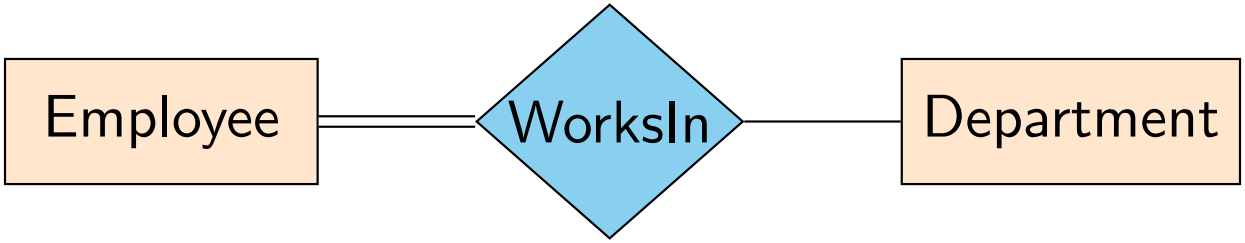
Participation Constraints

Participation constraints specify whether an entity's existence depends on its being related to another entity.

- **Partial Participation (Optional):** Not every entity in the set must participate in the relationship. This is the default.
- **Total Participation (Mandatory):** Every entity in the set **must** participate in the relationship.

Notation: The Double Line

A double line '=' from an entity set to a relationship indicates **total participation**.



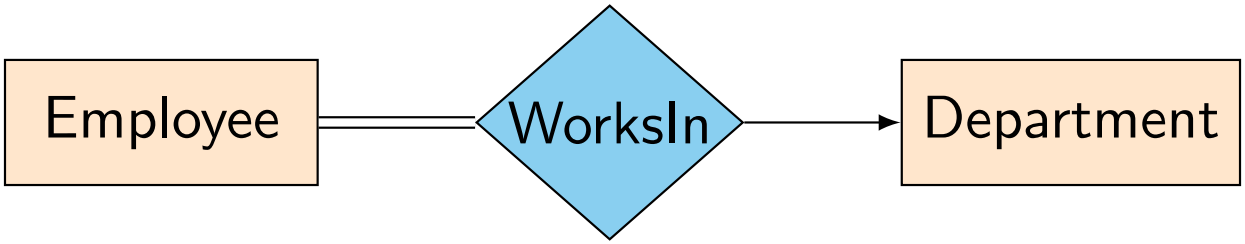
(Every employee MUST work in a department. Not every department must have an employee.)

Combining Cardinality and Participation

These two types of constraints are combined to precisely model business rules.

Rule: "Every employee must work in *exactly one* department."

- "Every employee..." $\implies$ Total participation of 'Employee'.
- "...in exactly one department" $\implies$ Arrow to 'Department'.



Notation: The Rounded Arrow

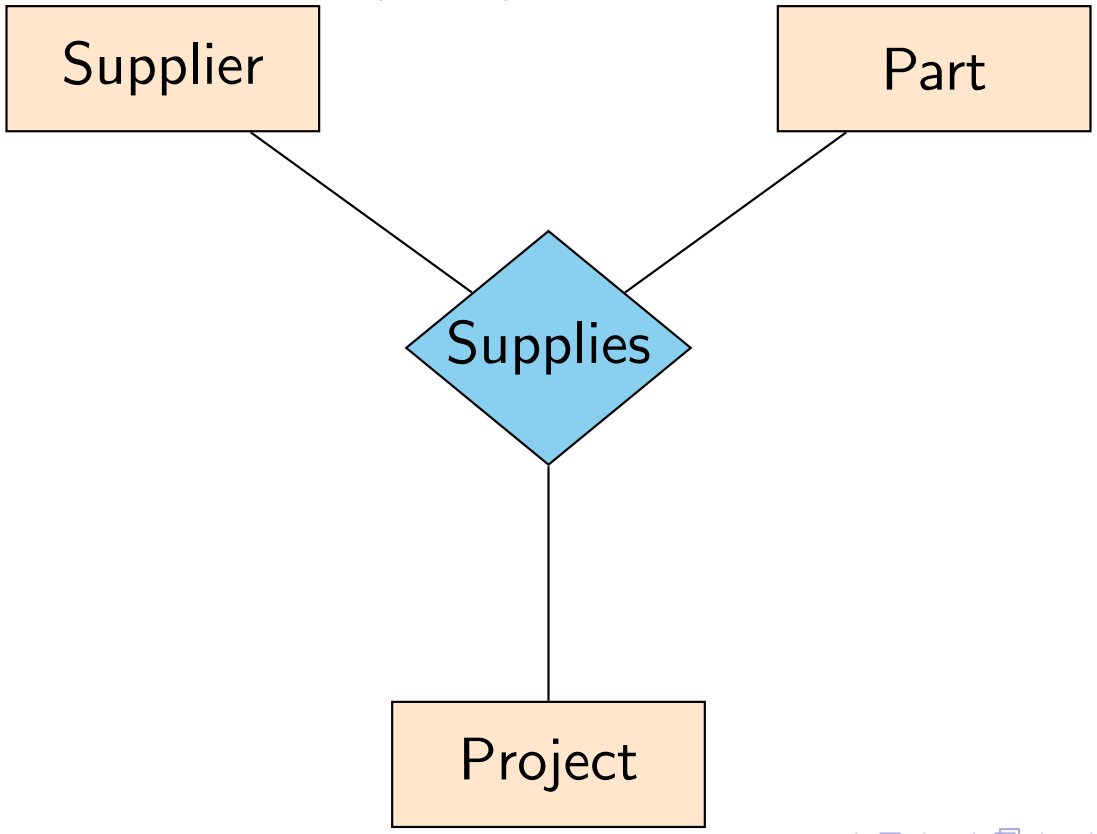
Some notations use a rounded arrow to represent "exactly one," which combines total participation and a 1-cardinality constraint. It's a useful shorthand.

3. Advanced E/R: Ternary Relationships

Ternary Relationships (Degree 3)

Sometimes, a relationship is an indivisible association between three entities.

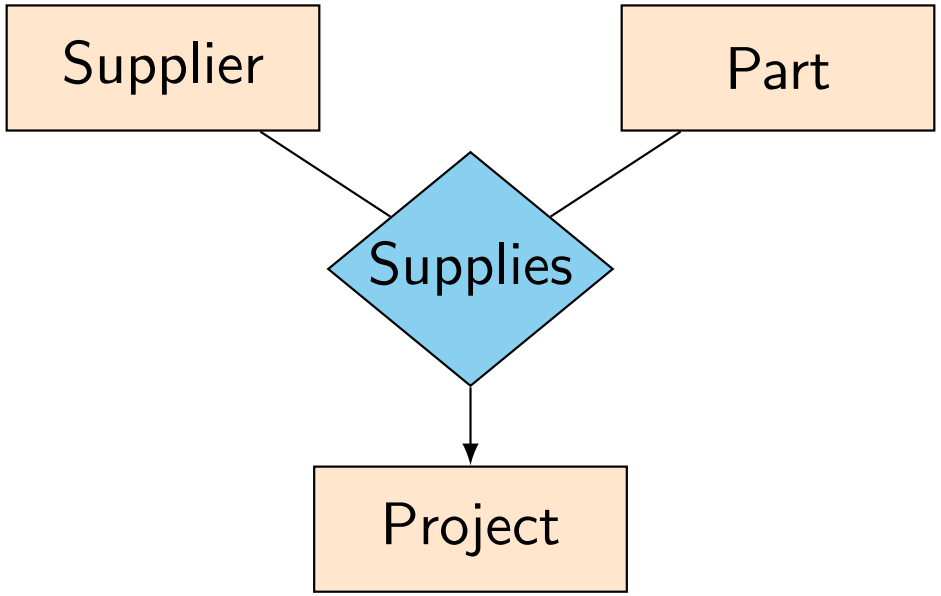
Scenario: We need to track which suppliers provide which parts to which projects. The relationship is the combination of all three. A supplier might provide a part, but not to every project.



Constraints on Ternary Relationships

Arrows on multi-way relationships are more complex. An arrow to entity set E means:

> *For any given combination of entities from all other participating sets, they are related to* **at most one** *entity from E.*



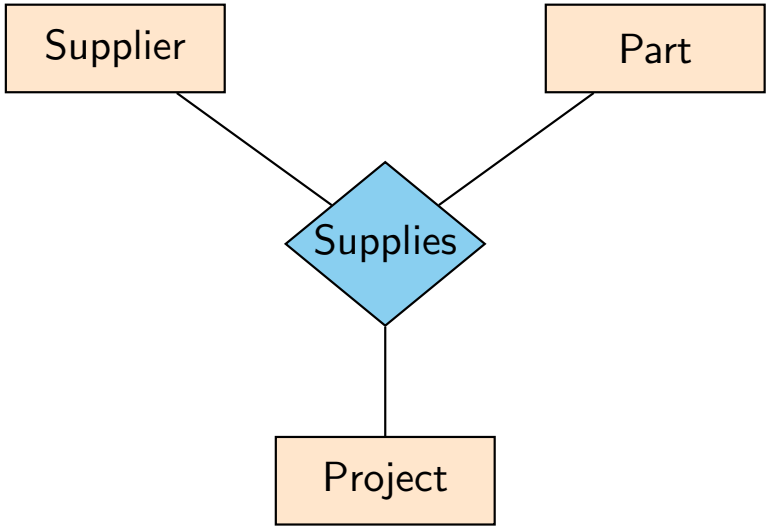
Interpretation of the Arrow

For any given (*Supplier*, *Part*) pair, they can supply to at most one *Project*. This is a very strong constraint!

Design Pitfall: When to Avoid Ternary Relationships

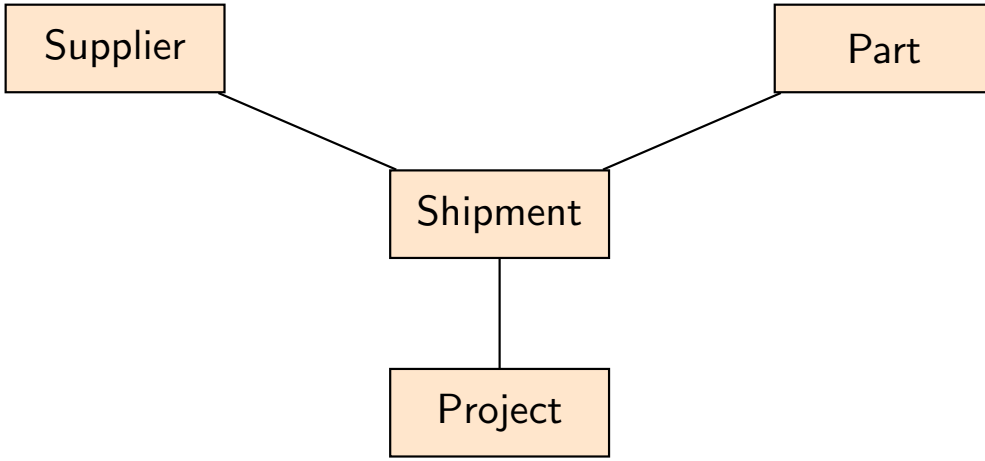
Ternary (and higher-degree) relationships can be difficult to interpret and constrain. It is often better practice to convert them into an associative entity.

Ternary Model



- Hard to add attributes (e.g., quantity).
- Constraints are ambiguous.

Associative Entity Model



- Easy to add attributes (e.g., shipmentID, quantity, date).
- Constraints are now standard binary constraints.

Case Study: Hospital Treatment

Business Rules:

1. We need to track treatments given to patients.
2. Every treatment involves one ‘Patient‘, one ‘Physician‘, and one ‘Diagnosis‘.
3. For a given patient and physician, they may have multiple diagnoses over time.
4. We must record the ‘date‘ of each treatment.

Option A: Ternary Relationship

Model this with a single ‘Treatment‘ relationship connecting the three entities. Where would the ‘date‘ attribute go?

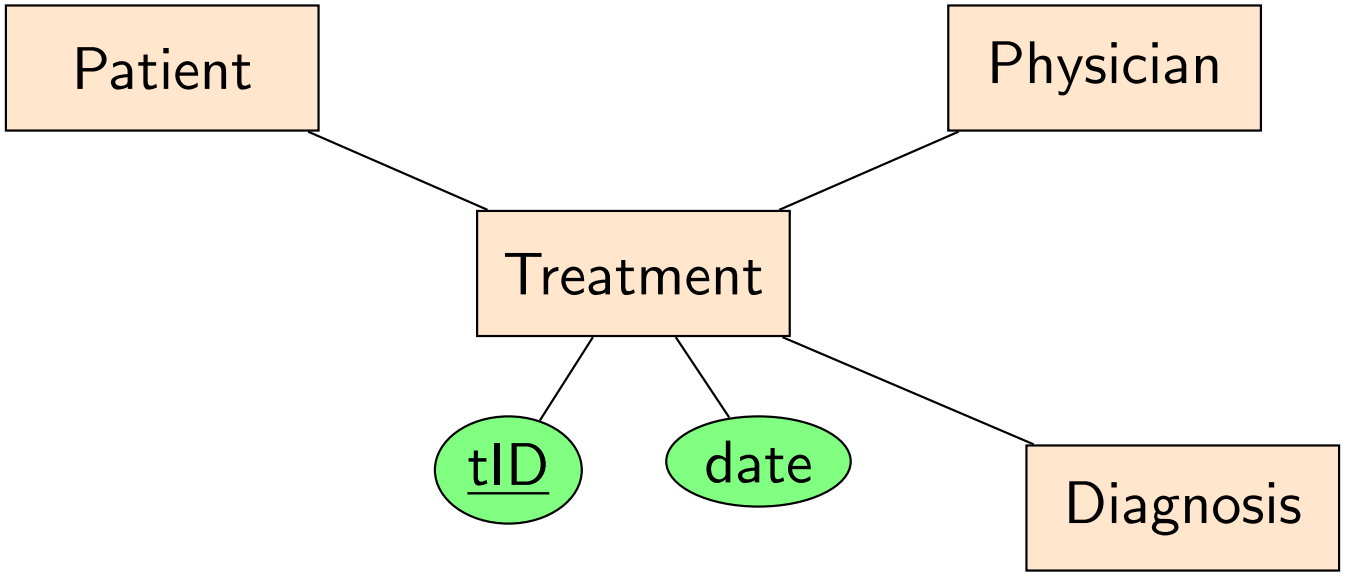
Option B: Associative Entity

Model this with a ‘Treatment‘ entity set connected to the other three via binary relationships.

Question: Which model best captures the business rules, and why?

Case Study Solution & Discussion

Option B (Associative Entity) is superior.



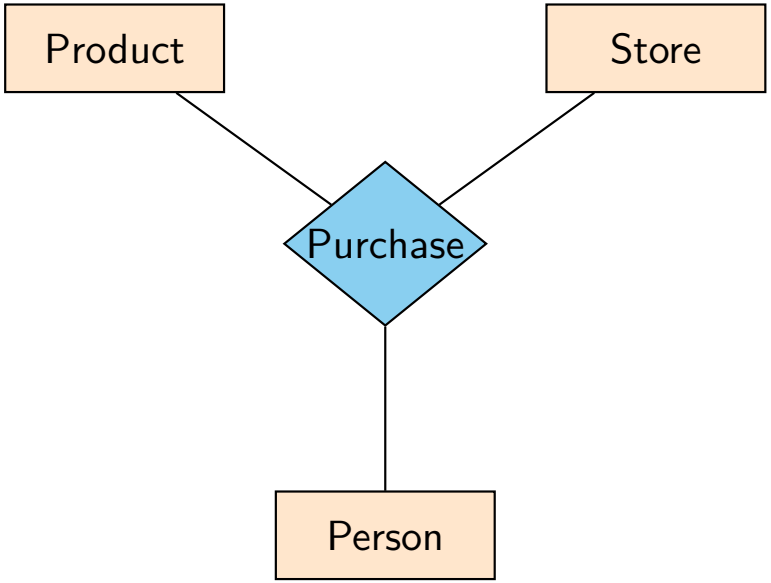
Justification

- **Handles Multiplicity:** Rule #3 states that a patient-physician pair can have multiple interactions. An associative entity with its own key (tID) allows for multiple treatment instances.
- **Attribute Placement:** The 'date' attribute is a natural fit for an attribute on the 'Treatment' entity.
- **Flexibility:** This model is far more extensible.

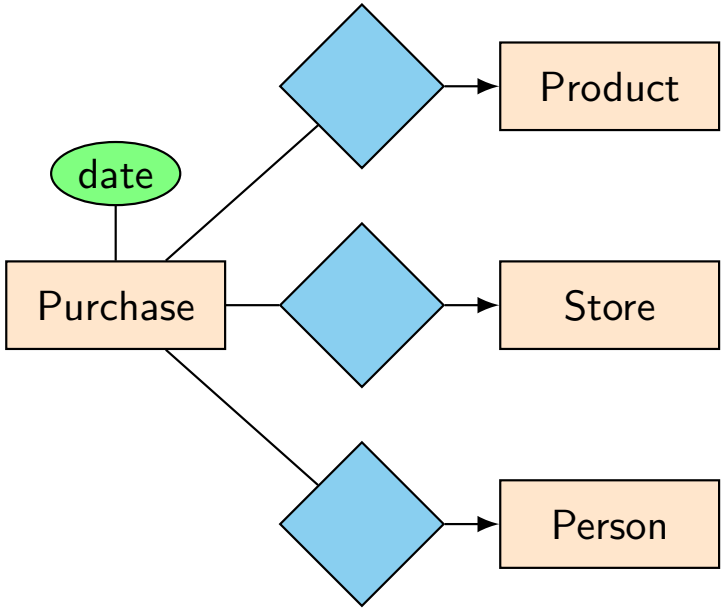
Decomposing Multi-way Relationships

Ternary relationships are often less flexible and harder to manage than a set of binary relationships. A common design pattern is to decompose them.

Ternary Model



Associative Entity Model

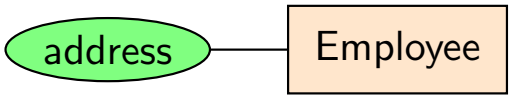


Key Insight: The decomposition introduces a new central entity (‘Purchase‘) and three **many-to-one** relationships. Each ‘Purchase‘ instance is linked to exactly one ‘Product‘, one ‘Store‘, and one ‘Person‘.

Design Guideline: Entity vs. Attribute

Scenario: Should an employee's 'address' be a simple attribute or its own entity?

Model as Attribute(s)



+ Simple, direct.
- What if an employee has multiple addresses?
- What if we want to query on components (e.g., city, zip)?
- What if multiple employees share the same address?

Model as an Entity

+ Can handle multiple addresses (many-to-many relationship).
+ Address can have its own attributes (street, city, zip).
+ Reduces redundancy if addresses are shared.

Rule of Thumb Promote: a concept from an attribute to an entity if it has its own descriptive properties, if it can be shared, or if multiple values of it need to be stored for a single entity.

4. From E/R Diagram to Relational Schema

Translating E/R Models to Relations

The Core Algorithm

The process of converting an E/R diagram into a set of relational tables (a schema) is systematic.

1. Translate strong entity sets.
2. Translate relationship sets.
3. (Later) Translate weak entity sets and subclass structures.
4. (Later) Apply schema refinements and merge relations where appropriate.

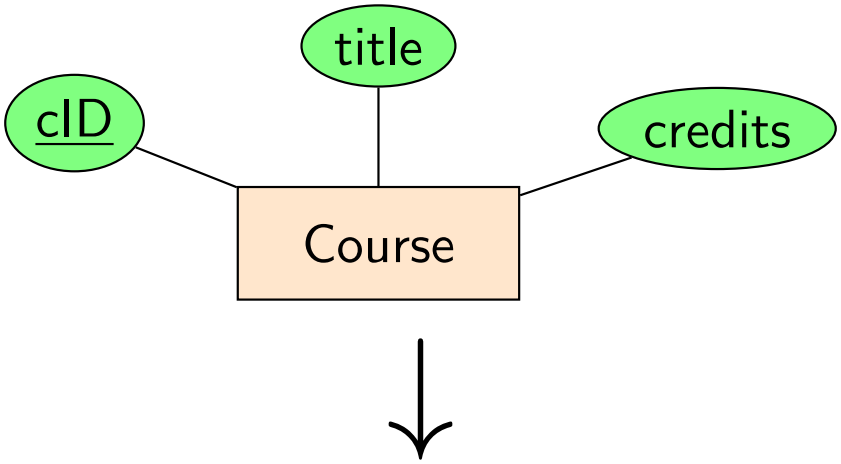
Fundamental Mapping

Both entity sets and relationship sets will become relations (tables). The attributes of the E/R construct become the columns of the table.

Step 1: Translating Strong Entity Sets

For each strong entity set, create a new relation (table).

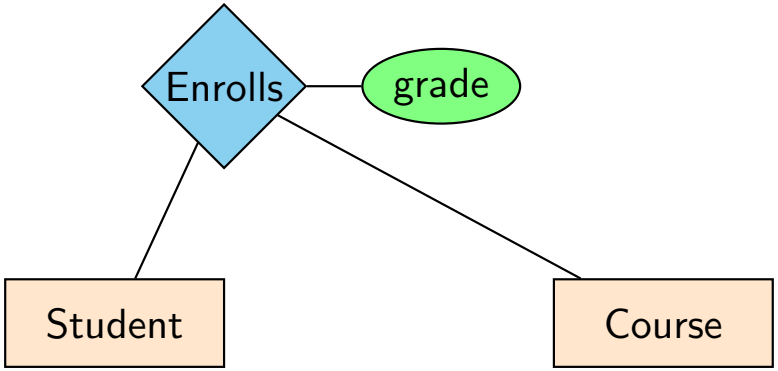
- The attributes of the entity set become the columns of the relation.
- The primary key of the entity set becomes the primary key of the relation.




```
CREATE TABLE Course (
    cID      VARCHAR(10) PRIMARY KEY,
    title    VARCHAR(100) NOT NULL,
    credits  INT
);
```

Step 2: Translating Relationship Sets

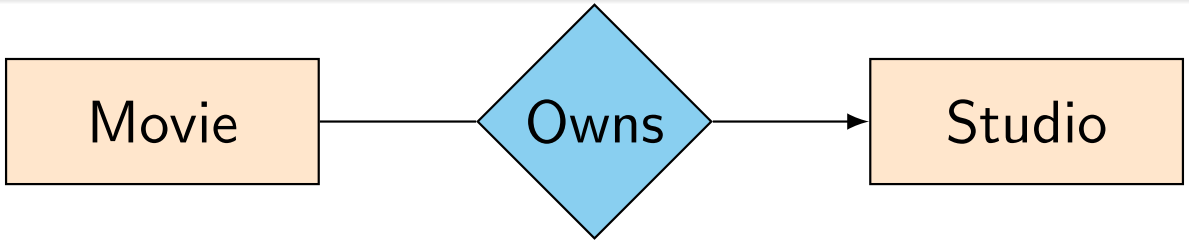
- The columns are the union of the primary keys from all participating entity sets.
- Any attributes on the relationship also become columns.
- The primary key of this new relation is the combination of all the entity keys.
- Each entity key column becomes a foreign key referencing its respective entity table.




```sql
CREATE TABLE Enrolls (
    studentID    VARCHAR(10),
    courseID     VARCHAR(10),
    grade        CHAR(1),
    PRIMARY KEY (studentID, courseID
   ),
    FOREIGN KEY (studentID)
   REFERENCES Student(sID),
    FOREIGN KEY (courseID)
   REFERENCES Course(cID)
);
```

Schema Refinement: Combining Relations

Optimization Opportunity: In a **many-to-one** relationship from E to F, the relation for entity set E and the relation for relationship set R can be merged.



(A movie is owned by at most one studio)

Initial Translation (2 Tables):

- 'Movie(title, year, ...)'
- 'Owns(movieTitle, movieYear, studioName)'

Combined Translation (1 Table):

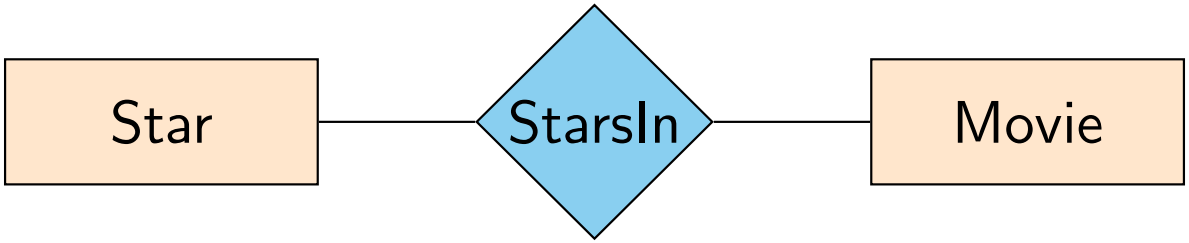
- 'Movie(title, year, ..., studioName)'

Benefit and Mechanism

This avoids a join operation. The 'studioName' is simply added to the 'Movie' table as an attribute that is also a foreign key referencing 'Studio'.

Why Merging Only Works for Many-to-One

Question: What happens if we try to merge a **many-to-many** relationship?



If we add a ‘starName‘ column to the ‘Movie‘ table, what do we do when a movie has multiple stars?

- Storing a list of stars in one field violates the First Normal Form (1NF) of relational databases.
- Creating multiple rows for the same movie (one for each star) introduces massive data redundancy and update anomalies.

Merging is only a valid optimization for many-to-one and one-to-one relationships. Many-to-many relationships **must** be represented by their own separate table.

5. Advanced E/R Constructs

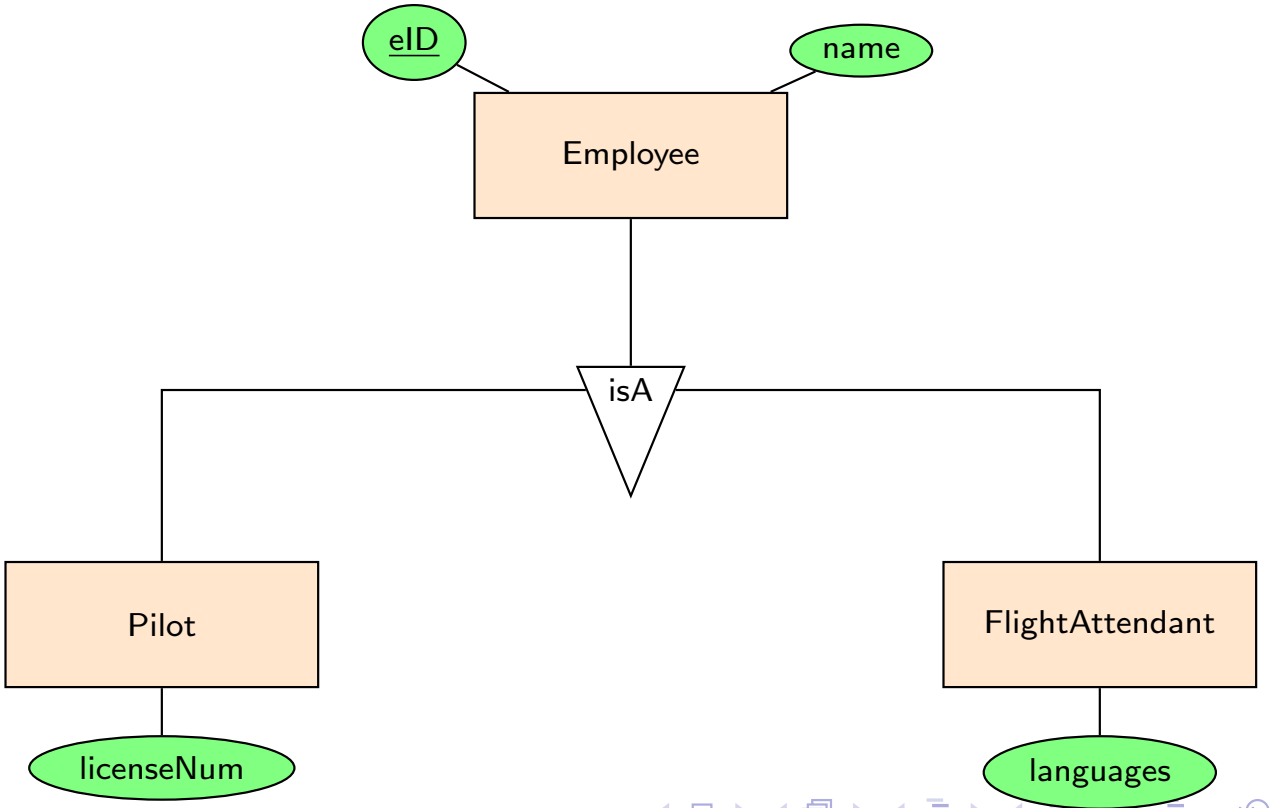
Specialization and Generalization (isA)

The **isA** relationship models a superclass/subclass hierarchy.

- The superclass is the **generalization**.
- The subclasses are **specializations**.
- Denoted by a triangle pointing to the superclass.

Attribute Inheritance

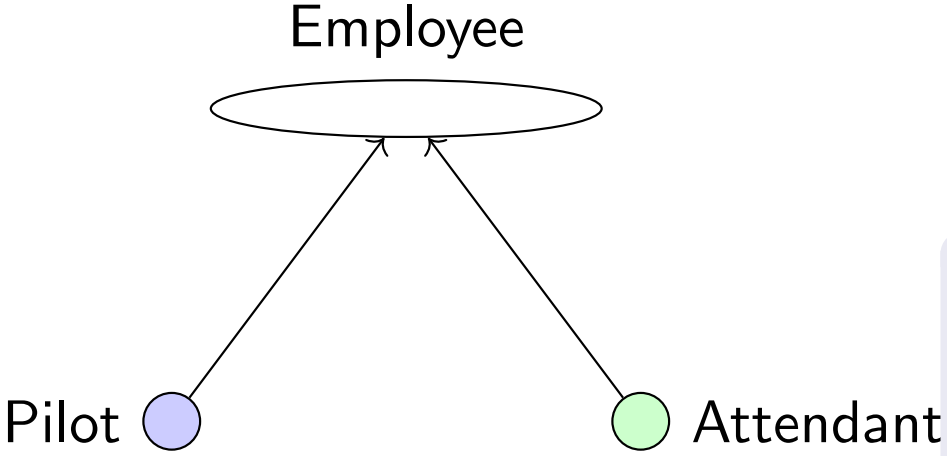
Entities in a subclass inherit all attributes of their superclass(es). They can also have their own specific attributes.



E/R Inheritance vs. OO Inheritance

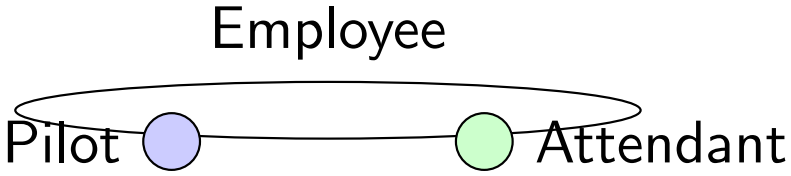
There is a critical difference between the E/R model's 'isA' and class inheritance in languages like Java or C++.

Object-Oriented Inheritance



Sets of objects are **disjoint**. An object is either a 'Pilot' or an 'Attendant', but not both.

E/R 'isA' Relationship



Entity sets can **overlap**. An employee entity can be both a 'Pilot' and a 'FlightAttendant' simultaneously.

Weak Entity Sets

A **weak entity set** is one whose entities cannot be uniquely identified by their own attributes alone. They must be associated with an **owner** or **identifying entity set** via an **identifying relationship**.

Classic Example: University Buildings and Rooms

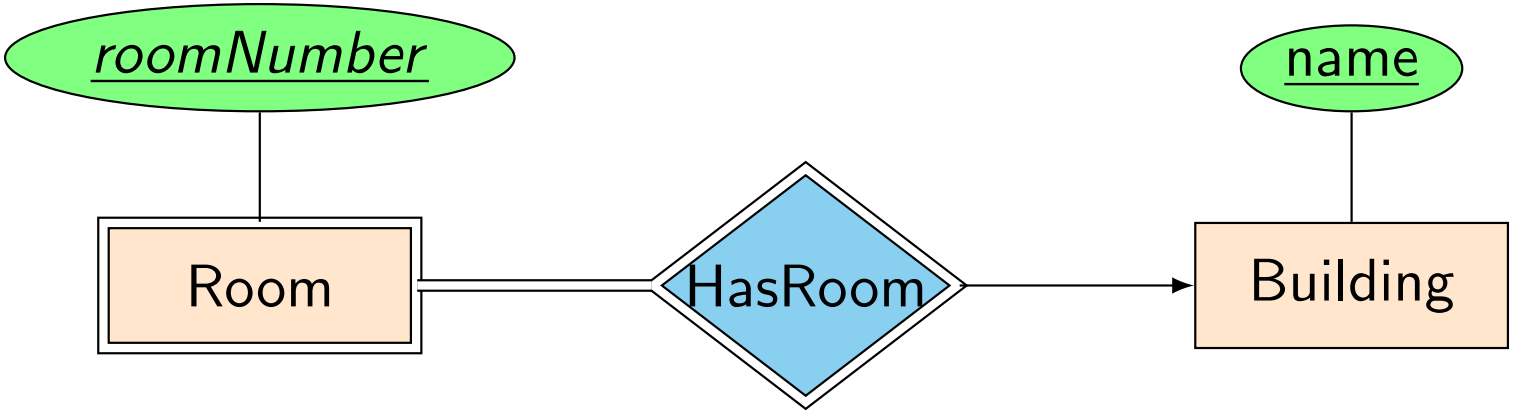
A ‘Room‘ entity cannot be identified by its ‘roomNumber‘ alone (e.g., room 101 exists in many buildings). It is only unique in the context of its ‘Building‘.

- **Weak Entity Set:** ‘Room‘. **Owner Entity Set:** ‘Building‘
- **Identifying Relationship:** ‘HasRoom‘
- **Partial Key / Discriminator:** ‘roomNumber‘ (underlined with a dashed line).

The full primary key for a weak entity is the combination of its own partial key and the primary key of its owner entity.
Key for ‘Room‘ = {<u>buildingName</u>, <u>roomNumber</u>}

Weak Entity Set Notation

- A weak entity set is shown as a rectangle with a **double border**.
- The identifying relationship is a diamond with a **double border**.
- The relationship from the weak set to its owner must be many-to-one, and the weak set must have total participation.



Translating Weak Entity Sets

A weak entity set and its identifying relationship are translated into a **single table**.

- The columns are the attributes of the weak entity set, plus the primary key of the owner entity set.
- The primary key of the table is the combination of the owner's key and the weak entity's partial key.
- The owner's key in this new table acts as a foreign key.

```sql
CREATE TABLE Room (
    roomNumber     VARCHAR(10),
    buildingName   VARCHAR(50),
    capacity       INT,
    PRIMARY KEY (buildingName, roomNumber),
    FOREIGN KEY (buildingName) REFERENCES Building(name)
        ON DELETE CASCADE
);
```

This ensures that if a 'Building' is deleted, all its associated 'Room' records are also automatically deleted.

Summary

- Conceptual modeling with E/R diagrams is a critical, implementation-agnostic first step in database design.
- Every design choice (entity vs. attribute, relationship degree, constraints) reflects a specific business rule. There are always trade-offs.
- The translation from a conceptual E/R diagram to a logical relational schema is a systematic, rule-based process.
- Advanced concepts like specialization and weak entities allow us to model more complex, real-world domains with high fidelity.

Next: Normalization

After we have an initial relational schema from our E/R diagram, normalization is a formal process we use to analyze and improve the schema to minimize redundancy and prevent update anomalies.